BÁO CÁO TOÀN BỘ DỰ ÁN — PRIORITY SCHEDULING (OS)

Thông tin chung

- Nhóm: 10
- Mã học phần: 012012500105
- GV hướng dẫn: Mai Ngọc Châu
- Đề tài: Mô phỏng lập lịch CPU theo độ ưu tiên

---

1) Mục tiêu dự án

Dự án xây dựng chương trình C++/Qt6 mô phỏng và trực quan hóa **Priority Scheduling** với hai biến thể trên **cùng một tập dữ liệu CSV**:

1. **Non-preemptive Priority**: khi tiến trình được cấp CPU thì chạy đến hết Burst Time.
2. **Preemptive Priority**: nếu tiến trình mới đến có **ưu tiên cao hơn** (priority nhỏ hơn) thì **giành CPU ngay**.

Chương trình có các yêu cầu chính:

- Đọc tiến trình từ CSV: `pid, arrival, burst, priority`.
- Tính các chỉ số thời gian: **CT, TAT, WT, RT**.
- Sinh đầu ra CSV (NP/P hoặc cả hai).
- Hiển thị kết quả trên GUI: bảng kết quả, so sánh trung bình và **Gantt chart**.
- Tối ưu để GUI có thể hiển thị tối đa cỡ **~1.000.000 dòng** nhờ **virtual table**.

---

2) Luật ràng buộc & tie-break

2.1 Validation dữ liệu đầu vào (CsvHandler)

- `pid` là số nguyên > 0, **duy nhất**.
- `arrival` là `long long` ≥ 0.
- `burst` là `long long` > 0.
- `priority` là `int`, **priority nhỏ hơn = ưu tiên cao hơn**.

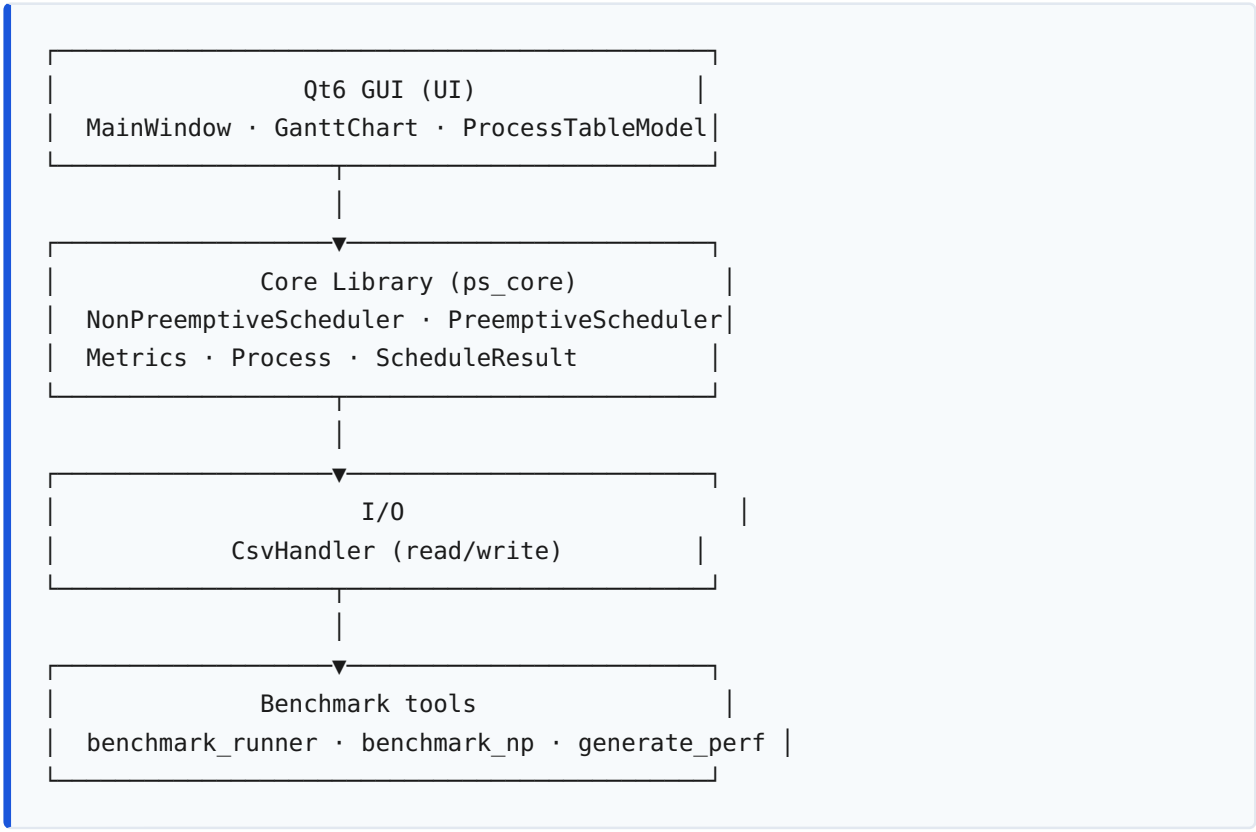
2.2 Tie-break (áp dụng cho cả NP và P)

- Nếu nhiều tiến trình có cùng `priority`:
- `arrival` nhỏ hơn được chọn trước.
- Nếu `arrival` bằng nhau: chọn theo **thứ tự xuất hiện trong CSV** (vị trí trong vector input).

---

3) Thiết kế hệ thống (kiến trúc lớp)

Hệ thống chia thành các lớp rõ ràng để dễ kiểm thử:



Quy tắc phụ thuộc:

- `core` không phụ thuộc Qt/I/O.
- `io` không phụ thuộc Qt.
- `ui` có thể phụ thuộc cả core và io.

---

4) Triển khai chính

4.1 Non-preemptive Priority (NP)

Tư tưởng:

- Sắp xếp index theo `(arrival, inputIndex)`.
- Duy trì hàng đợi sẵn sàng theo priority bằng cấu trúc heap/min-heap (kèm tie-break để giữ đúng thứ tự input).
- Khi CPU rảnh: nếu ready queue rỗng thì chèn đoạn **IDLE** đến arrival kế tiếp.

4.2 Preemptive Priority (P)

Tư tưởng tối ưu so với phiên bản ở checkpoint trước:

- Dùng **mô phỏng theo sự kiện (event-based)** (nhảy tới lần arrival tiếp theo hoặc thời điểm chuyển ngữ cảnh quan trọng), không chạy tick-by-tick.
- Ready queue là cấu trúc min-heap theo `(priority, arrival, inputIndex)`.
- Khi có tiến trình mới đến với priority cao hơn thì cập nhật thời điểm chuyển và **gộp đoạn Gantt** liên tiếp cùng PID.

Nhờ event-based, hiệu năng Preemptive được cải thiện rõ rệt.

4.3 GUI tối ưu dữ liệu lớn

- Bảng kết quả dùng **`QTableView` + model ảo (`ProcessTableModel`)**.
- Chỉ render vùng nhìn thấy → không bị treo do số lượng dòng cực lớn.
- Gantt chart được bọc trong `QScrollArea` (cuộn **ngang**), vẽ theo vùng `event->rect()` để giữ mượt khi cuộn.

---

5) Kiểm thử

Toàn bộ test dùng `doctest`, chạy qua `ctest` thành 3 nhóm:

Suite	Test cases	Assertions	Kết quả
unit_tests	79	112,207	Pass
integration_tests	10	318	Pass
gui_tests	10	215	Pass
Tổng	**99**	**112,740**	**ctest 3/3 bộ pass**

---

6) Benchmark hiệu năng

6.1 Non-preemptive Priority (NP)

- 100,000: **13.7 ms**
- 1,000,000: **242 ms**
- 2,000,000: **577 ms**

6.2 Preemptive Priority (P)

- 100,000: **27 ms**
- 500,000: **242 ms**
- 1,000,000: **572 ms**

6.3 GUI với 1.000.000 dòng

Máy đo (offscreen, 1180×900):

- Load CSV: **272 ms**
- Chạy NP only: **251 ms**
- Chạy NP+P: **835 ms**
- Peak RSS: **247 MiB**

---

7) Kết quả & nhận xét

- Preemptive cho RT/WT trung bình thường tốt hơn nhờ giành CPU khi có tiến trình ưu tiên cao hơn đến.
- Số segment Gantt có thể tăng do preemption.
- GUI xử lý dữ liệu lớn tốt nhờ virtual table và Gantt vẽ theo vùng nhìn.

---

8) Kết luận

Dự án hoàn thành yêu cầu so sánh **Non-preemptive vs Preemptive Priority**:

1. **Đúng đắn**: đã chạy `ctest` **3/3 bộ** với tổng **99 test cases / 112,740 assertions**.
2. **Hiệu năng**: NP và P đạt các mốc thời gian mong đợi, đặc biệt P được tối ưu nhờ event-based.
3. **Khả năng mở rộng GUI**: hiển thị được cỡ **1.000.000 dòng** mà vẫn ổn định.
4. **Tính tái lập**: benchmark có cấu trúc rõ, sinh dữ liệu và ghi log phục vụ đối chiếu.

---

9) Phân công thành viên (tóm tắt)

Thành viên	Phần việc chính
Hưng (Leader)	Kiến trúc, tích hợp, QA, benchmark, tài liệu
Kiệt (NP)	Non-preemptive scheduler + unit tests
Vĩ (P)	Preemptive scheduler + unit tests
Minh Hưng (GUI)	MainWindow/Gantt/virtual table + gui tests
Lan (CSV)	CsvHandler + validate + integration IO

Phụ lục A — Hướng dẫn build & chạy

```
sudo apt install qt6-base-dev cmake g++ git
git clone https://github.com/Hhanguu/PriorityScheduling.git
cd PriorityScheduling
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j4

# Chạy ứng dụng
./build/PriorityScheduling

# Chạy test
QT_QPA_PLATFORM=offscreen ctest --test-dir build --output-on-failure
```

Phụ lục B — Invariants (bảo toàn)

- Kết quả sắp xếp giữ đúng thứ tự input cho các trường hợp tie-break.
- Gantt cover đầy đủ `[0, max(CT)]`, không chồng lấn và không có đoạn rỗng bất thường.
- Merge các segment liền kề cùng PID.
- Metrics (WT/TAT/RT) được tính nhất quán theo công thức.